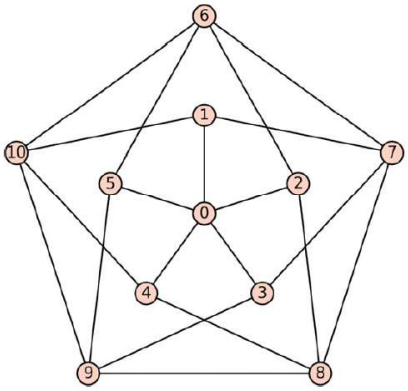
CSE 5004-Graph Theory and Algorithms

Exercise - 1

1. Fill in the blanks for the following Grötzsch graph.



	VALUE		TRUE/FALSE
Vertice Count	**11**	Is it regular?	**FALSE**
Edge Count	**20**	Is it directed?	**FALSE**
Radius	**2**	Is it connected?	**TRUE**
Girth	**4**	Is it weighted?	**FALSE**
Diameter	**2**		

2. For the graph with the adjacency matrix below, find the
(a) adjacency list,
(b) incidence matrix and
(c) a drawing of it.

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 & 0 \end{bmatrix}$$

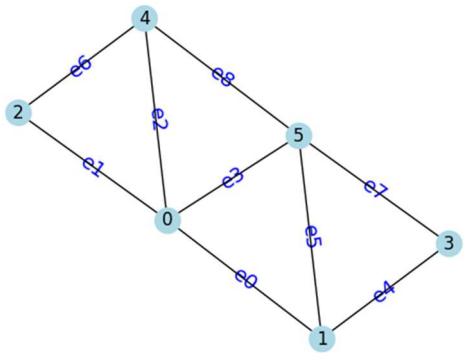
(a) Adjacency List

Node	*Neighbors*
0	1, 2, 4, 5
1	0, 3, 5
2	0, 4
3	1, 5
4	0, 2, 5
5	0, 1, 3, 5

(b) Incidence Matrix

	e0	*e1*	*e2*	*e3*	*e4*	*e5*	*e6*	*e7*	*e8*
0	**1**	**1**	**1**	**1**	0	0	0	0	0
1	**1**	0	0	0	**1**	**1**	0	0	0
2	0	**1**	0	0	0	0	**1**	0	0
3	0	0	0	0	**1**	0	0	**1**	0
4	0	0	**1**	0	0	0	**1**	0	**1**
5	0	0	0	**1**	0	**1**	0	1	**1**

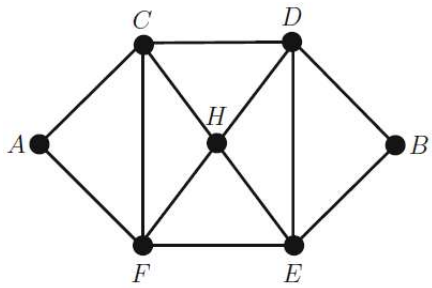
(c) Drawing of Graph



CSE 5004-Graph Theory and Algorithms

Exercise - 1

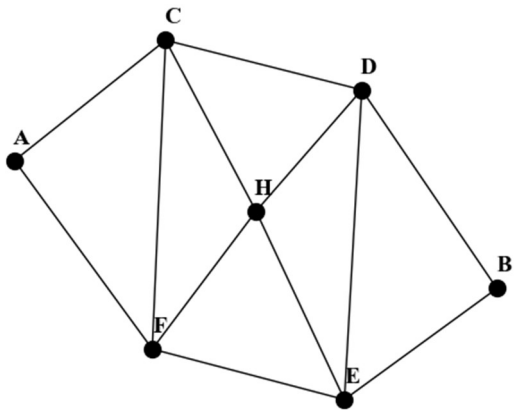
3.



Modify the following Python code to visualize the graph above. Paste the resulting graph drawing.

```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.Graph()
edges = [
    ('A', 'C'), ('A', 'F'), ('F', 'H'), ('C', 'D'),
    ('C', 'F'), ('C', 'H'), ('D', 'H'), ('D', 'E'),
    ('D', 'B'), ('E', 'F'), ('E', 'B'), ('E', 'H')]
for edge in edges:
    G.add_edge(*edge)

graph_pos = nx.spring_layout(G)
label_pos = {node: [pos[0] + 0.03, pos[1] + 0.04]
             for node, pos in graph_pos.items()}
nx.draw_networkx(
    G, pos=graph_pos, node_color='black',
    node_size=100, with_labels=False)
nx.draw_networkx_labels(
    G, font_size=14, font_family='Times New Roman',
    font_weight='bold', pos=label_pos)

plt.show()
```

Resulting Graph:



CSE 5004-Graph Theory and Algorithms

Exercise - 1

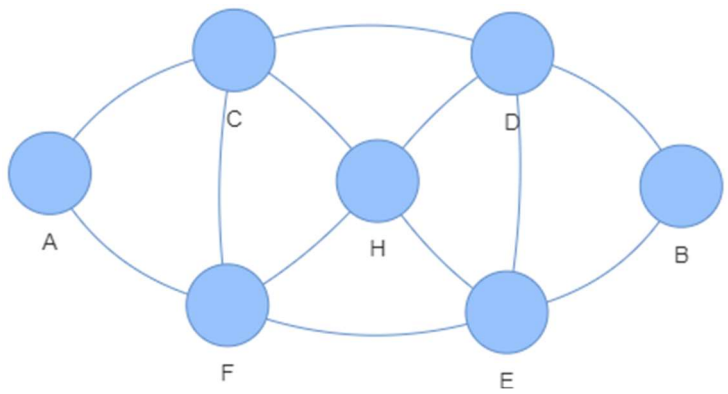
4. Calculate the following properties for the graph given in question 3.

```
nx.eccentricity(G) == {'A':3, 'C':2, 'F':2, 'H':2, 'D':2, 'E':2, 'B':3}
nx.diameter(G) == 3
nx.radius(G) == 2
nx.periphery(G) == ['A', 'B']
nx.center(G) == ['C', 'F', 'H', 'D', 'E']
```

5. Modify the following interactive network graph to visualize the graph in question 3. Paste the resulting graph drawing.

```
from pyvis import network as net

G = net.Network(height='400px', notebook=True, cdn_resources='remote')
nodes = ['A', 'B', 'C', 'D', 'E', 'F', 'H']
edges = [
    ('A', 'C'), ('A', 'F'), ('F', 'H'), ('C', 'D'),
    ('C', 'F'), ('C', 'H'), ('D', 'H'), ('D', 'E'),
    ('D', 'B'), ('E', 'F'), ('E', 'B'), ('E', 'H')]
for node in nodes:
    G.add_node(node)
for edge in edges:
    G.add_edge(*edge)

G.show('mygraph.html')
```

Resulting Graph:

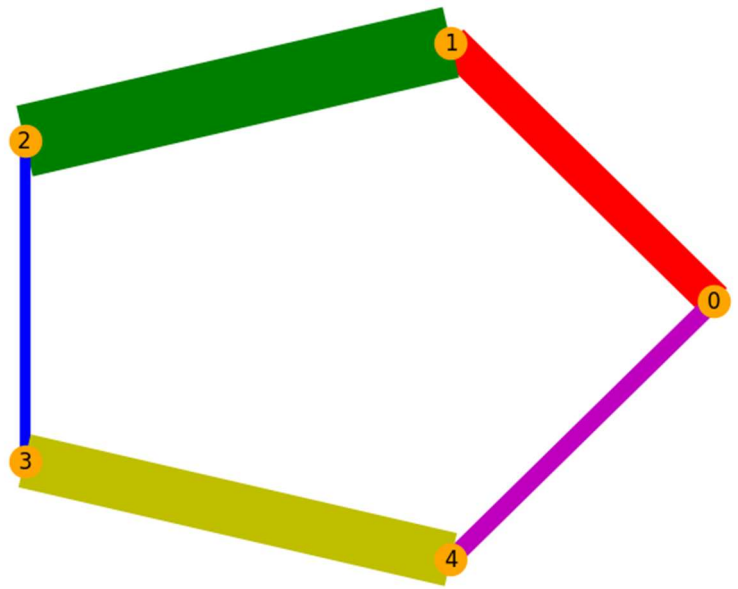


6. Execute the following Python code. Paste the resulting graph drawing.

```
import networkx as nx

G = nx.Graph()
G.add_edge(0,1,color='r',weight=20)
G.add_edge(1,2,color='g',weight=40)
G.add_edge(2,3,color='b',weight=6)
```

CSE 5004-Graph Theory and Algorithms

Exercise - 1

```
G.add_edge(3,4,color='y',weight=30)
G.add_edge(4,0,color='m',weight=10)

colors = nx.get_edge_attributes(G,'color').values()
weights = nx.get_edge_attributes(G,'weight').values()

pos = nx.circular_layout(G)
nx.draw(G, pos,
        edge_color=colors,
        width=list(weights),
        with_labels=True,
        node_color='orange')
```

Resulting Graph: